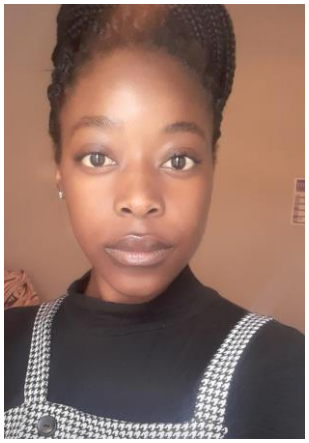
TOPIC: CLINIC APPOINTMENT SYSTEM

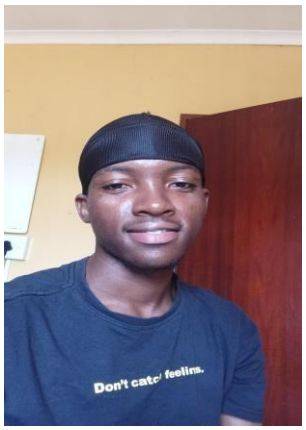
GROUP12 PROJECT REPORT

19 MAY 2025

GROUP MEMBER INFORMATION

Surnames & Initials	Student Number	Picture	Contact Information
Mr. Mbombi RB **(Group Leader)**	40676099		Cell: +27 66 539 1680 Email: 40676099@mynwu.ac.za
Mr. Madonsela R	39803384		Cell: +27 79 277 9959 Email: 39803384@mynwu.ac.za

Ms. Mukhumo B	38880199		Cell: +27 72 388 3167 Email: 38880199@mynwu.ac.za
Ms. Masinga MS	44527802		Cell: +27 73 913 6169 Email: 44527802@mynwu.ac.za
Ms. Munyai MG	42551900		Cell: +27 72 110 5380 Email: 42551900@mynwu.ac.za
Mr. Chauke DX	42705614		Cell: +27 79 867 8786 Email: 42705614@mynwu.ac.za

Mr. Khoza S	32462409		Cell: +27 76 602 6847 Email: 32462409@mynwu.ac.za
Mr. Mabasa NV	47337354		Cell: +27 67 636 8768 Email: ntshembho17@gmail.com

EXECUTIVE SUMMARY

Project Purpose:

To develop a database system that streamlines clinic operations by automating appointments, managing patient records, and improving service efficiency while ensuring data security and compliance.

Key Objectives:

1. **Efficient Scheduling:** Prevent double bookings and optimize doctor availability.
2. **Patient Management:** Securely store and track medical records and appointment history.
3. **Automation:** Reduce missed appointments with reminders and simplify administrative tasks.
4. **Compliance:** Ensure data security (POPIA) and integrate with existing systems.
5. **Scalability:** Support clinic growth with a user-friendly, expandable system.

This system aims to enhance patient care, reduce wait times, and improve operational efficiency.

DATABASE (INITIAL STUDY)

COMPANY OVERVIEW.

A multi-specialty healthcare clinic offering medical services, diagnostics, and outpatient care. It aims to improve efficiency through a digital appointment and patient record system

OBJECTIVES, OPERATIONS, AND ORGANIZATIONAL STRUCTURE

Objectives:

- Enable online appointment booking.
- Help doctors manage availability and schedules.
- Store and manage patient data securely.
- Reduce wait times and scheduling conflicts.
- Automate resource and billing management.

Operations:

- Online appointment scheduling and cancellations.
- Doctor schedule management.
- Automated reminders via SMS/email.
- Patient medical record management.
- Waitlist and availability notifications.
- Secure login and role-based access.
- Personalized user dashboards.

Organizational Structure:

- Leadership: Clinic Director (CEO), Medical Director, Operations Manager.
- Support Teams: Finance, HR/Admin, IT, Maintenance.
- Medical Staff: GPs, Specialists, Nurses, Lab Technicians, Pharmacists.
- Frontline Staff: Receptionists, IT Support.

IDENTIFIED PROBLEMS AND CONSTRAINTS

Problems:

- Manual scheduling errors and double bookings.
- Limited access to doctor availability.
- No centralized patient history.
- Paper-based records are error-prone.
- Outdated tech and low outreach.

Constraints:

- Real-time updates for changing doctor schedules.
- Limited tech skills among staff.
- Budget and legacy system limitations.
- Need for user-friendly, scalable, low-disruption implementation.

DATABASE SYSTEM SPECIFICATIONS

Main Goals:

- Shorter wait times.

- Better patient data management.
- Automated billing and reminders.
- Support for online consultations.
- Follow legal rules (POPIA).
- Easy and fast to use.

Information Needed:

- **Patients**: ID, name, contact, medical history.
- **Appointments**: ID, date/time, doctor, notes.
- **Staff**: ID, name, role, schedule.
- **Medical Records**: Diagnosis, treatments, tests.
- **Pharmacy**: Medicine name, stock, expiry.
- **Billing**: Invoice, amount, payment status.

What's Included:

- Patient records and appointments.
- Medical and billing information.
- Inventory and staff schedules.
- Reports and summaries.

What's Not Included:

- Emergency services.
- Staff payroll.
- Smart (AI) diagnosis.

Boundaries:

- Only the right people can access data (based on role).
- Data is secure and backed up.
- Works fast and handles growth.
- Made for one clinic or a small clinic group

BUSINESS RULES:

Patient Management Rules

- Each patient must have a unique Patient ID.
- A patient must provide full name, date of birth, contact details, and address upon registration.
- A patient's medical history, allergies, and insurance information must be stored securely.
- A patient can only view their own medical records through the secure patient portal.
- Patients can be placed on a waitlist if no appointment slots are available and will be notified when a slot opens.

Appointment Scheduling Rules

- Each appointment must have a unique Appointment ID.
- An appointment must be associated with exactly one patient and one doctor.
- Appointments must have a status (Scheduled, Completed, or Canceled).
- Appointments cannot be double-booked — no overlapping times for the same doctor or patient.
- Appointments can be booked, rescheduled, or canceled by patients or staff, depending on access rights.
- Doctors must define their availability in the system for appointments to be scheduled.
- System must automatically prevent scheduling outside a doctor's defined availability.

- Notifications must be sent (via SMS/email) for all booked, rescheduled, or canceled appointments.

Medical Staff Rules

- Each staff member (doctor, nurse) must have a unique Staff ID.
- Staff must be assigned a role (e.g., GP, Specialist, Nurse) and their contact and schedule must be stored.
- Doctors can only access and update consultation data for patients they are treating.
- Only medical staff can add/edit diagnosis, treatment plans, and prescriptions.

Medical Records Rules

- Each medical record must have a unique Record ID.
- Medical records must link to a specific patient and the doctor who handled the consultation.
- The system must support uploading/storing lab results, prescriptions, and treatment plans.
- Medical history and consultation notes must be timestamped and immutable (i.e., cannot be edited after finalization, unless by admin with logs).

Pharmacy and Inventory Rules

- Each medication or supply must have a unique Item ID.
- The system must track stock quantity, expiry date, and supplier info.
- Alerts must be generated when stock levels fall below the reorder level.
- Only pharmacy staff can update inventory records.

Billing & Payment Rules

- Each invoice must have a unique Invoice ID.
- Each invoice must be linked to one patient and must include total amount, payment method, and payment status.
- Invoices can be updated only by finance staff or administrators.
- Patients can view their own billing history but cannot modify payment records.

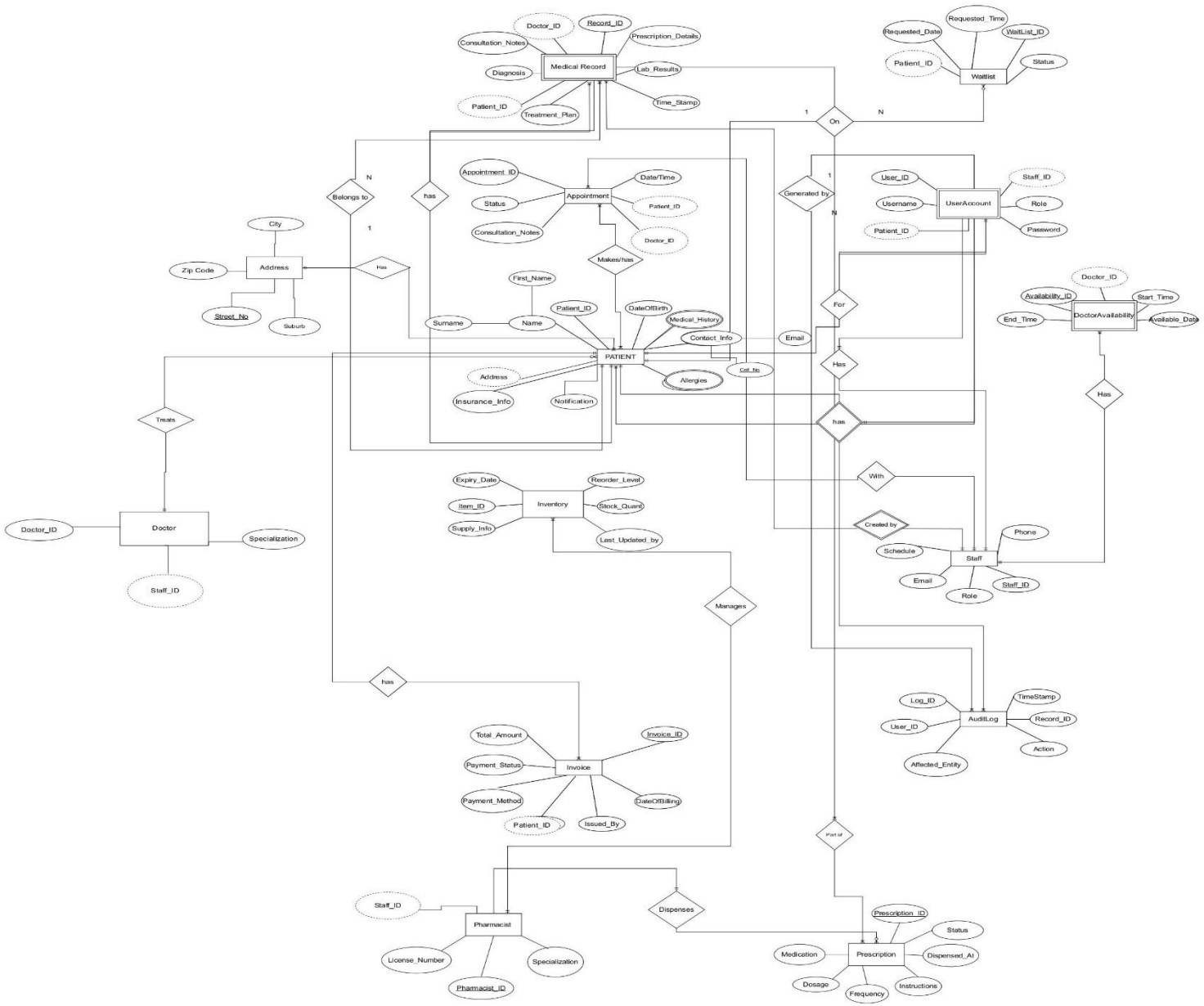
System Access and Security Rules

- All users must authenticate with a secure login based on their role: Admin, Doctor, Nurse, Receptionist, or Patient.
- Role-Based Access Control (RBAC) must be enforced — users can only access data and functions permitted to their role.
- All sensitive data (medical, financial, personal) must be encrypted in storage and during transmission.
- System must maintain audit logs for all data access, updates, and deletions.
- Regular data backups must be conducted and stored securely.

Compliance and Regulatory Rules

- All patient and staff data must comply with POPIA (Protection of Personal Information Act).
- Any data breach or unauthorized access attempt must be logged and reported to administrators immediately.

The Entity Relationship Diagram (ERD)



Entities and Attributes:

Patient

- Patient_ID (PK)
- First_Name
- Surname
- DateOfBirth
- Cell_No
- Email
- Medical_History
- Allergies
- Contact_Info
- Insurance_Info
- Notification

Staff

- Staff_ID (PK)
- Phone
- Email
- Schedule
- Role
- Address
- Address_ID (PK)
- Patient_ID (FK → Patient)
- Street
- Suburb
- City
- Zip_Code
- Doctor (Specialization of Staff)
- Doctor_ID (PK)
- Staff_ID (FK → Staff, unique)
- Specialization

Pharmacist (Specialization of Staff)

- Pharmacist_ID (PK)
- Staff_ID (FK → Staff, unique)
- License_Number
- Specialization
- Appointment
- Appointment_ID (PK)
- Patient_ID (FK → Patient)
- Doctor_ID (FK → Doctor)
- DateTime
- Consultation_Notes
- Status

Medical_Record

- Record_ID (PK)
- Patient_ID (FK → Patient)
- Doctor_ID (FK → Doctor)
- Lab_Results
- Treatment_Plan
- Diagnosis

- Consultation_Notes
- Time_Stamp

Prescription

- Prescription_ID (PK)
- Record_ID (FK → Medical_Record)
- Medication
- Dosage
- Frequency
- Instructions
- Dispensed_At
- Status
- Last_Updated_By (FK → Staff)

Invoice

- Invoice_ID (PK)
- Patient_ID (FK → Patient)
- Total_Amount
- Payment_Method
- Payment_Status
- DateOfBilling
- Issued_By (FK → Staff)

Inventory

- Item_ID (PK)
- Stock_Quant
- Reorder_Level
- Expiry_Date
- Supply_Info

UserAccount

- User_ID (PK)
- Staff_ID (FK → Staff)
- Username (unique)
- Password

AuditLog

- Log_ID (PK)
- User_ID (FK → UserAccount)
- Action
- Affected_Entity
- TimeStamp

Waitlist

- Waitlist_ID (PK)
- Patient_ID (FK → Patient)
- Doctor_ID (FK → Doctor)
- Requested_Date
- Requested_Time
- Status
- DoctorAvailability
- Availability_ID (PK)
- Doctor_ID (FK → Doctor)
- Available_Date
- Start_Time
- End_Time

Relationships:

1. **Patient → Address**: One-to-One (1 patient has 1 address)
2. **Staff → Doctor/Pharmacist**: Specialization (Inheritance)
   A Doctor is a Staff member (1:1 via Staff_ID)
   A Pharmacist is a Staff member (1:1 via Staff_ID)
3. **Patient → Appointment**: One-to-Many (1 patient can have many appointments)
4. **Doctor → Appointment**: One-to-Many (1 doctor can have many appointments)
5. **Patient → Medical_Record**: One-to-Many (1 patient can have many records)
6. **Doctor → Medical_Record**: One-to-Many (1 doctor can create many records)
7. **Medical_Record → Prescription**: One-to-Many (1 record can have many prescriptions)
8. **Staff → Prescription**: One-to-Many (1 staff member can update many prescriptions)
9. **Patient → Invoice**: One-to-Many (1 patient can have many invoices)
10. **Staff → Invoice**: One-to-Many (1 staff member can issue many invoices)
11. **Staff → UserAccount**: One-to-One (1 staff member has 1 user account)
12. **UserAccount → AuditLog**: One-to-Many (1 user can trigger many audit logs)
13. **Patient → Waitlist**: One-to-Many (1 patient can be on many waitlists)
14. **Doctor → Waitlist**: One-to-Many (1 doctor can have many waitlist entries)
15. **Doctor → DoctorAvailability**: One-to-Many (1 doctor can have many availability slots)

PHYSICAL DESIGN

1. Tables, data types and constraints

```
CREATE TABLE Patient (
  Patient_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  First_Name VARCHAR(50) NOT NULL,
  Surname VARCHAR(50) NOT NULL,
  DateOfBirth DATE NOT NULL CHECK (DateOfBirth < CURRENT_DATE),
  Cell_No VARCHAR(15) NOT NULL UNIQUE,
  Email VARCHAR(100) UNIQUE CHECK (Email LIKE '%@%.%'),
  Medical_History TEXT,
  Allergies TEXT,
  Contact_Info VARCHAR(255),
  Insurance_Info VARCHAR(255),
  Notification BOOLEAN DEFAULT FALSE,
  Created_At TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  Updated_At TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE Staff (
  Staff_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Phone VARCHAR(15) NOT NULL UNIQUE CHECK (Phone ~ '^[0-9]+$'),
  Email VARCHAR(100) NOT NULL UNIQUE CHECK (Email LIKE '%@%.%'),
  Schedule TEXT,
  Role VARCHAR(50) NOT NULL CHECK (Role IN ('Doctor', 'Pharmacist', 'Nurse', 'Receptionist',
'Admin')),
  Active BOOLEAN DEFAULT TRUE,
  Created_At TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE Address (
  Address_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Patient_ID INT NOT NULL,
  Street VARCHAR(100) NOT NULL,
  Suburb VARCHAR(50) NOT NULL,
  City VARCHAR(50) NOT NULL,
  Zip_Code VARCHAR(10) NOT NULL,
  Coordinates GEOGRAPHY(POINT, 4326),
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID) ON DELETE CASCADE,
  CONSTRAINT unique_patient_address UNIQUE (Patient_ID)
);
CREATE TABLE Doctor (
  Doctor_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Staff_ID INT NOT NULL UNIQUE,
  Specialization VARCHAR(100) NOT NULL,
  License_Number VARCHAR(50) NOT NULL UNIQUE,
  FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID) ON DELETE CASCADE
);
CREATE TABLE Pharmacist (
  Pharmacist_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Staff_ID INT NOT NULL UNIQUE,
  License_Number VARCHAR(50) NOT NULL UNIQUE,
  Specialization VARCHAR(100),
  FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID) ON DELETE CASCADE
);
CREATE TABLE Appointment (
  Appointment_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Patient_ID INT NOT NULL,
  Doctor_ID INT NOT NULL,
  DateTime TIMESTAMP NOT NULL CHECK (DateTime > CURRENT_TIMESTAMP),
  Duration INTERVAL DEFAULT '30 minutes',
  Consultation_Notes TEXT,
  Status VARCHAR(20) NOT NULL CHECK (Status IN ('Scheduled', 'Completed', 'Cancelled', 'No-
Show')),
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID) ON DELETE CASCADE,
  FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE CASCADE,
  CONSTRAINT no_overlapping_appointments EXCLUDE USING gist (
    Doctor_ID WITH =,
    tsrange(DateTime, DateTime + Duration) WITH &&
  )
);
CREATE TABLE Medical_Record (
  Record_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Patient_ID INT NOT NULL,
  Doctor_ID INT NOT NULL,
  Lab_Results JSONB,
  Treatment_Plan TEXT,
  Diagnosis VARCHAR(255)[],
  Consultation_Notes TEXT,
  Version INT DEFAULT 1,
  Time_Stamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID) ON DELETE CASCADE,
```

```
  FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE SET NULL
);
CREATE TABLE Prescription (
  Prescription_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Record_ID INT NOT NULL,
  Medication VARCHAR(100) NOT NULL CHECK (Medication ~ '^[A-Z][a-zA-Z0-9 ]+$'),
  Dosage VARCHAR(50) NOT NULL,
  Frequency VARCHAR(50) NOT NULL,
  Instructions TEXT,
  Dispensed_At TIMESTAMP,
  Status VARCHAR(20) NOT NULL CHECK (Status IN ('Pending', 'Dispensed', 'Cancelled', 'Expired')),
  Last_Updated_By INT NOT NULL,
  FOREIGN KEY (Record_ID) REFERENCES Medical_Record(Record_ID) ON DELETE CASCADE,
  FOREIGN KEY (Last_Updated_By) REFERENCES Staff(Staff_ID) ON DELETE SET NULL,
  CONSTRAINT valid_prescription CHECK (
    (Status = 'Dispensed' AND Dispensed_At IS NOT NULL) OR
    (Status != 'Dispensed' AND Dispensed_At IS NULL)
  )
);
CREATE TABLE Invoice (
  Invoice_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Patient_ID INT NOT NULL,
  Total_Amount DECIMAL(10, 2) NOT NULL CHECK (Total_Amount >= 0),
  Tax_Amount DECIMAL(10, 2) GENERATED ALWAYS AS (Total_Amount * 0.15) STORED,
  Payment_Method VARCHAR(50) CHECK (Payment_Method IN ('Cash', 'Credit Card', 'Medical Aid',
'Bank Transfer')),
  Payment_Status VARCHAR(20) NOT NULL CHECK (Payment_Status IN ('Paid', 'Unpaid', 'Pending',
'Partially Paid')),
  DateOfBilling DATE NOT NULL DEFAULT CURRENT_DATE,
  Issued_By INT NOT NULL,
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID) ON DELETE CASCADE,
  FOREIGN KEY (Issued_By) REFERENCES Staff(Staff_ID) ON DELETE SET NULL
);
CREATE TABLE Inventory (
  Item_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Description TEXT,
  Stock_Quant INT NOT NULL CHECK (Stock_Quant >= 0),
  Reorder_Level INT NOT NULL CHECK (Reorder_Level >= 0),
  Unit_Price DECIMAL(10, 2) CHECK (Unit_Price >= 0),
  Expiry_Date DATE CHECK (Expiry_Date > CURRENT_DATE),
  Supplier_Info JSONB,
  Last_Restock_Date DATE,
  CONSTRAINT positive_inventory CHECK (Stock_Quant >= Reorder_Level OR Reorder_Level = 0)
);
CREATE TABLE UserAccount (
  User_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Staff_ID INT NOT NULL UNIQUE,
  Username VARCHAR(50) NOT NULL UNIQUE CHECK (LENGTH(Username) >= 5),
  Password_Hash VARCHAR(255) NOT NULL,
  Last_Login TIMESTAMP WITH TIME ZONE,
  Is_Active BOOLEAN DEFAULT TRUE,
```

```
  Failed_Attempts INT DEFAULT 0,
  FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID) ON DELETE CASCADE
);
CREATE TABLE AuditLog (
  Log_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  User_ID INT NOT NULL,
  Action_Type VARCHAR(50) NOT NULL CHECK (Action_Type IN ('INSERT', 'UPDATE', 'DELETE', 'LOGIN',
'LOGOUT')),
  Affected_Table VARCHAR(50) NOT NULL,
  Record_ID INT,
  Old_Values JSONB,
  New_Values JSONB,
  TimeStamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  IP_Address INET,
  FOREIGN KEY (User_ID) REFERENCES UserAccount(User_ID) ON DELETE CASCADE
);
CREATE TABLE Waitlist (
  Waitlist_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Patient_ID INT NOT NULL,
  Doctor_ID INT NOT NULL,
  Requested_Date DATE NOT NULL,
  Priority INT DEFAULT 5 CHECK (Priority BETWEEN 1 AND 10),
  Status VARCHAR(20) NOT NULL CHECK (Status IN ('Waiting', 'Confirmed', 'Cancelled')),
  Created_At TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID) ON DELETE CASCADE,
  FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE CASCADE
);
CREATE TABLE DoctorAvailability (
  Availability_ID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Doctor_ID INT NOT NULL,
  Available_Date DATE NOT NULL,
  Start_Time TIME NOT NULL,
  End_Time TIME NOT NULL CHECK (End_Time > Start_Time),
  Slot_Duration INTERVAL DEFAULT '30 minutes',
  Is_Recurring BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID) ON DELETE CASCADE,
  CONSTRAINT no_overlapping_availability EXCLUDE USING gist (
    Doctor_ID WITH =,
    tsrange(
      (Available_Date + Start_Time),
      (Available_Date + End_Time)
    ) WITH &&
  )
);
```

2. **Indexes**

-- Patient indexes

```
CREATE INDEX idx_patient_name ON Patient (Surname, First_Name);
CREATE INDEX idx_patient_contact ON Patient (Cell_No, Email);
```

-- Appointment indexes

```
CREATE INDEX idx_appointment_datetime ON Appointment (DateTime);
CREATE INDEX idx_appointment_patient ON Appointment (Patient_ID);
CREATE INDEX idx_appointment_doctor ON Appointment (Doctor_ID, Status);
```

-- Medical record indexes

```
CREATE INDEX idx_medical_record_patient ON Medical_Record (Patient_ID);
CREATE INDEX idx_medical_record_diagnosis ON Medical_Record USING GIN (Diagnosis);
```

-- Prescription indexes

```
CREATE INDEX idx_prescription_medication ON Prescription (Medication);
CREATE INDEX idx_prescription_status ON Prescription (Status, Dispensed_At);
```

-- Invoice indexes

```
CREATE INDEX idx_invoice_patient ON Invoice (Patient_ID);
CREATE INDEX idx_invoice_status ON Invoice (Payment_Status, DateOfBilling);
```

-- Full-text search index

```
CREATE INDEX idx_staff_search ON Staff USING GIN (to_tsvector('english', Role || ' ' || Email));
```

-- Geospatial index for addresses

```
CREATE INDEX idx_address_geo ON Address USING GIST (Coordinates);
```

3. Views for common queries

-- Active patient directory

```
CREATE VIEW vw_active_patients AS
SELECT p.Patient_ID, p.First_Name, p.Surname, p.Cell_No, p.Email,
    a.Street, a.City, a.Zip_Code,
    COUNT(DISTINCT app.Appointment_ID) AS total_appointments
FROM Patient p
JOIN Address a ON p.Patient_ID = a.Patient_ID
LEFT JOIN Appointment app ON p.Patient_ID = app.Patient_ID
WHERE p.Notification = TRUE
GROUP BY p.Patient_ID, a.Address_ID;
```

-- Doctor schedule view

```
CREATE VIEW vw_doctor_schedule AS
SELECT d.Doctor_ID, s.First_Name || ' ' || s.Surname AS Doctor_Name,
    a.DateTime AS Appointment_Time,
    p.First_Name || ' ' || p.Surname AS Patient_Name,
    a.Status, a.Consultation_Notes
FROM Doctor d
JOIN Staff s ON d.Staff_ID = s.Staff_ID
JOIN Appointment a ON d.Doctor_ID = a.Doctor_ID
JOIN Patient p ON a.Patient_ID = p.Patient_ID
WHERE a.DateTime >= CURRENT_DATE;
```

-- Prescription tracking view

```
CREATE VIEW vw_prescription_status AS
SELECT pr.Prescription_ID, p.First_Name || ' ' || p.Surname AS Patient,
    pr.Medication, pr.Dosage, pr.Status,
    mr.Diagnosis, s.First_Name || ' ' || s.Surname AS Prescribed_By,
    pr.Dispensed_At
FROM Prescription pr
JOIN Medical_Record mr ON pr.Record_ID = mr.Record_ID
JOIN Patient p ON mr.Patient_ID = p.Patient_ID
JOIN Doctor d ON mr.Doctor_ID = d.Doctor_ID
JOIN Staff s ON d.Staff_ID = s.Staff_ID;
```

-- Financial summary view

```
CREATE VIEW vw_financial_summary AS
SELECT
  EXTRACT(YEAR FROM DateOfBilling) AS Year,
  EXTRACT(MONTH FROM DateOfBilling) AS Month,
  Payment_Method,
  COUNT(*) AS Invoice_Count,
  SUM(Total_Amount) AS Total_Revenue,
  SUM(CASE WHEN Payment_Status = 'Paid' THEN Total_Amount ELSE 0 END) AS Paid_Amount,
  SUM(CASE WHEN Payment_Status != 'Paid' THEN Total_Amount ELSE 0 END) AS Outstanding
FROM Invoice
GROUP BY Year, Month, Payment_Method;
```

-- Inventory alert view

```
CREATE VIEW vw_inventory_alerts AS
SELECT Item_ID, Name, Stock_Quant, Reorder_Level, Unit_Price,
    CASE
      WHEN Stock_Quant = 0 THEN 'Out of Stock'
      WHEN Stock_Quant <= Reorder_Level THEN 'Reorder Needed'
      ELSE 'In Stock'
    END AS Status,
    Expiry_Date
FROM Inventory
WHERE Stock_Quant <= Reorder_Level OR Expiry_Date < CURRENT_DATE + INTERVAL '30 days';
```

4. Populated data

```
INSERT INTO Patient VALUES
(1, 'Alice', 'Smith', '1990-05-20', '0712345678', 'alice@example.com', 'Hypertension', 'Penicillin', '123 Main St', 'MediCare123', TRUE),
(2, 'Bob', 'Johnson', '1985-11-10', '0723456789', 'bob@example.com', 'Diabetes', 'None', '456 West Ave', 'HealthPlus', FALSE),
(3, 'Clara', 'Ngwenya', '1992-08-14', '0734567890', 'clara@example.com', 'Asthma', 'Peanuts', '789 Elm Rd', 'BestHealth', TRUE),
(4, 'Daniel', 'Mokoena', '1979-01-05', '0745678901', 'daniel@example.com', 'None', 'Latex', '12 Long St', 'SA MedAid', FALSE),
(5, 'Evelyn', 'Khumalo', '2000-12-25', '0756789012', 'evelyn@example.com', 'Migraines', 'Dust', '44 High Rise', 'LifeCover', TRUE);
```

```
INSERT INTO Staff VALUES
(101, '0781234567', 'dr.mokoena@clinic.com', 'Mon-Fri 9am-5pm', 'Doctor'),
(102, '0792345678', 'pharma.joy@clinic.com', 'Mon-Fri 8am-4pm', 'Pharmacist'),
(103, '0711112222', 'nurse.lee@clinic.com', 'Tue-Sat 9am-3pm', 'Nurse'),
(104, '0723334444', 'reception.zulu@clinic.com', 'Mon-Fri 7am-5pm', 'Receptionist'),
(105, '0735556666', 'admin.kim@clinic.com', 'Mon-Fri 8am-6pm', 'Admin');

INSERT INTO Address VALUES
(1, 1, '123 Main St', 'Sandton', 'Johannesburg', '2196'),
(2, 2, '456 West Ave', 'Arcadia', 'Pretoria', '0007'),
(3, 3, '789 Elm Rd', 'Morningside', 'Durban', '4001'),
(4, 4, '12 Long St', 'Claremont', 'Cape Town', '7708'),
(5, 5, '44 High Rise', 'Polokwane', 'Polokwane', '0699');

INSERT INTO Doctor VALUES
(201, 101, 'General Practitioner');

INSERT INTO Pharmacist VALUES
(301, 102, 'PH123456', 'Dispensing');

INSERT INTO Appointment VALUES
(1, 1, 201, '2025-05-15 10:00:00', 'Routine checkup', 'Scheduled'),
(2, 2, 201, '2025-05-15 11:00:00', 'Blood pressure review', 'Scheduled'),
(3, 3, 201, '2025-05-16 09:30:00', 'Asthma follow-up', 'Scheduled'),
(4, 4, 201, '2025-05-16 10:30:00', 'Annual physical', 'Scheduled'),
(5, 5, 201, '2025-05-17 14:00:00', 'Migraine consultation', 'Scheduled');

INSERT INTO Medical_Record VALUES
(1, 1, 201, 'Normal', 'Continue monitoring', 'Healthy', 'Routine checkup complete', '2025-05-15
10:45:00'),
(2, 2, 201, 'Elevated BP', 'Prescribe beta-blockers', 'Hypertension', 'Blood pressure concerns', '2025-05-
15 11:45:00'),
(3, 3, 201, 'Stable', 'Maintain inhaler use', 'Asthma', 'No issues reported', '2025-05-16 09:45:00'),
(4, 4, 201, 'Normal', 'No action needed', 'Healthy', 'Patient in good health', '2025-05-16 10:45:00'),
(5, 5, 201, 'MRI required', 'Refer to neurologist', 'Migraine', 'Recurring headaches', '2025-05-17
14:45:00');

INSERT INTO Prescription VALUES
(1, 2, 'Atenolol', '50mg', 'Once daily', 'Take after breakfast', '2025-05-15 12:00:00', 'Dispensed', 102),
(2, 3, 'Salbutamol Inhaler', '2 puffs', 'Twice daily', 'Use when wheezing', '2025-05-16 10:00:00',
'Dispensed', 102),
(3, 5, 'Ibuprofen', '400mg', 'Twice daily', 'Take with food', '2025-05-17 15:00:00', 'Pending', 102),
(4, 1, 'Multivitamin', '1 tablet', 'Daily', 'General health support', '2025-05-15 11:00:00', 'Dispensed', 102),
(5, 4, 'None', 'N/A', 'N/A', 'No medication needed', NULL, 'Not Required', 102);

INSERT INTO Invoice VALUES
(1, 1, 750.00, 'Credit Card', 'Paid', '2025-05-15', 105),
(2, 2, 500.00, 'Cash', 'Unpaid', '2025-05-15', 105),
(3, 3, 620.50, 'Medical Aid', 'Paid', '2025-05-16', 105),
(4, 4, 430.00, 'Credit Card', 'Paid', '2025-05-16', 105),
(5, 5, 999.99, 'Medical Aid', 'Pending', '2025-05-17', 105);
```

```
INSERT INTO Inventory VALUES
(1, 100, 20, '2026-12-31', 'HealthMed Suppliers'),
(2, 50, 10, '2025-10-01', 'PharmaLink'),
(3, 200, 30, '2026-01-15', 'MediCorp'),
(4, 0, 25, '2025-09-01', 'Global Meds'),
(5, 75, 15, '2026-06-20', 'CareLife Ltd');

INSERT INTO UserAccount VALUES
(1, 101, 'dr.mokoena', 'hashedpassword1'),
(2, 102, 'pharma.joy', 'hashedpassword2'),
(3, 103, 'nurse.lee', 'hashedpassword3'),
(4, 104, 'reception.zulu', 'hashedpassword4'),
(5, 105, 'admin.kim', 'hashedpassword5');

INSERT INTO AuditLog VALUES
(1, 1, 'Updated Consultation Notes', 'Medical_Record', '2025-05-15 11:00:00'),
(2, 2, 'Dispensed Medication', 'Prescription', '2025-05-15 12:05:00'),
(3, 3, 'Created Appointment', 'Appointment', '2025-05-14 09:30:00'),
(4, 5, 'Generated Invoice', 'Invoice', '2025-05-15 12:15:00'),
(5, 4, 'Scheduled Appointment', 'Appointment', '2025-05-14 10:00:00');

INSERT INTO Waitlist VALUES
(1, 2, 201, '2025-05-18', '09:00:00', 'Waiting'),
(2, 4, 201, '2025-05-18', '10:00:00', 'Waiting'),
(3, 5, 201, '2025-05-18', '11:00:00', 'Waiting'),
(4, 1, 201, '2025-05-18', '13:00:00', 'Cancelled'),
(5, 3, 201, '2025-05-18', '14:00:00', 'Confirmed');

INSERT INTO DoctorAvailability VALUES
(1, 201, '2025-05-15', '09:00:00', '17:00:00'),
(2, 201, '2025-05-16', '09:00:00', '17:00:00'),
(3, 201, '2025-05-17', '09:00:00', '17:00:00'),
(4, 201, '2025-05-18', '09:00:00', '15:00:00'),
(5, 201, '2025-05-19', '09:00:00', '17:00:00');
```

SQL QUERIES

1. List all patients with their contact and insurance info

```
SELECT First_Name, Surname, Cell_No, Email, Insurance_Info
FROM Patient;
```

2. List all appointments scheduled for a specific date

```
SELECT *
FROM Appointment
WHERE DATE(DateTime) = '2025-05-15';
```

3. Get full medical record for a specific patient

```
SELECT p.First_Name, p.Surname, m.*
FROM Patient p
JOIN Medical_Record m ON p.Patient_ID = m.Patient_ID
WHERE p.Patient_ID = 2;
```

4. Show first 3 patients only

```
SELECT Patient_ID, First_Name, Surname
FROM Patient
LIMIT 3;
```

5. List patients sorted by surname alphabetically

```
SELECT First_Name, Surname FROM Patient
ORDER BY Surname ASC;
```

6. Find patients with 'm' in their surname and allergies

```
SELECT * FROM Patient
WHERE Surname LIKE '%m%' AND Allergies IS NOT NULL;
```

7. UPPERCASE all patient surnames

```
SELECT UPPER(Surname) AS Capitalized_Surname FROM Patient;
```

8. Extract month of billing

```
SELECT Invoice_ID, MONTH(DateOfBilling) AS Billing_Month
FROM Invoice;
```

9. Total revenue generated

```
SELECT SUM(Total_Amount) AS Total_Revenue
FROM Invoice;
```

10. Group invoices by payment status and show total per status

```
SELECT Payment_Status, COUNT(*) AS Count, SUM(Total_Amount) AS Total
FROM Invoice
GROUP BY Payment_Status
HAVING Total > 600;
```

11. Join appointments with patient names

```
SELECT a.Appointment_ID, p.First_Name, p.Surname, a.DateTime
FROM Appointment a
JOIN Patient p ON a.Patient_ID = p.Patient_ID;
```

12. Get patients who have invoices above the average total

```
SELECT First_Name, Surname FROM Patient
WHERE Patient_ID IN (
  SELECT Patient_ID FROM Invoice
  WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Invoice)
);
```